

0.79.1. Suivre une autre lettre — de plus en plus vite

Le 0.79, mais le B accélère : toutes les 3 secondes, il attend un peu moins entre deux pas. Facile au début, de plus en plus dur.

1. Ce qu'il faut comprendre

C'est le 0.79, avec un B qui accélère. Au début, il est lent (600 ms par pas) : on lui échappe facilement. Toutes les 3 secondes, il devient un peu plus rapide, jusqu'à 200 ms par pas : là, il est presque impossible à fuir.

Ce qui est nouveau ici : une vitesse qui CHANGE pendant le jeu. `chaque(600)` ne peut pas le faire : son nombre est écrit en clair, traduit avant le jeu, et ne bouge plus. On compte donc les images nous-mêmes : `compte` augmente à chaque image, et quand il atteint `lenteur`, le B fait un pas.

`lenteur` est le nombre d'images entre deux pas : 36 au départ (600 ms). Toutes les 3 secondes, `if (chaque(3000))`, on lui retire 6 : 30, 24, 18... jusqu'à 12 (200 ms), le plus rapide. Le `if (lenteur > 12)` empêche de descendre plus bas.

`compte >= lenteur` (plus grand **ou égal**) : si `lenteur` diminue juste quand `compte` était déjà plus grand, le pas se fait quand même, au lieu d'attendre que `compte` fasse le tour des 255.

En bas à droite, `lenteur` s'affiche : on la voit descendre, 036, 030, 024...

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **2517 octets** de cartouche.

1. UNE VARIABLE — X

```
uint8_t x = 10;      // le A (la croix)
```

2. UNE VARIABLE — Y

```
uint8_t y = 8;
```

3. UNE VARIABLE — BX

```
uint8_t bx = 0;      // le B (le poursuivant)
```

4. UNE VARIABLE — BY

```
uint8_t by = 0;
```

5. UNE VARIABLE — SX

```
uint8_t sx = 0;      // la direction du prochain pas du B
```

6. UNE VARIABLE — SY

```
uint8_t sy = 0;
```

7. UNE VARIABLE — LENTEUR

```
uint8_t lenteur = 36; // les images entre deux pas du B : 36 = 600 ms au départ
```

8. UNE VARIABLE — COMPTE

```
uint8_t compte = 0; // les images comptées depuis son dernier pas
```

9. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    while (true) {
        image();

        // Toutes les 3 secondes, le B accélère : 6 images de moins entre deux pas.
        if (chaque(3000)) {
            if (lenteur > 12) { // 12 images (200 ms) : le plus rapide
                lenteur = lenteur - 6;
            }
        }

        // Le pas du B, quand compte atteint lenteur.
        compte++;
        if (compte >= lenteur) {
            compte = 0;
            sx = 0;
            if (x > bx) { sx = 1; }
            if (x < bx) { sx = -1; }
            sy = 0;
            if (y > by) { sy = 1; }
            if (y < by) { sy = -1; }
            if (sx != 0 || sy != 0) {
                un_pas(bx, by, ALPHABET[1], sx, sy);
            }
        }

        deplace_croix(x, y, ALPHABET[0], 250); // le A, par-dessus
        nombre(17, 17, lenteur); // la lenteur du B : elle descend

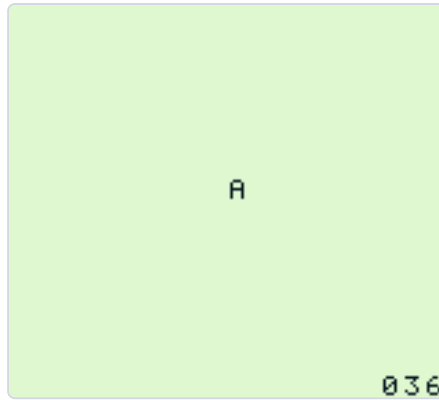
        if (bx == x && by == y) {
            texte(0, 17, "PRIS");
        }
    }
}
```

3. Ce qui se passe, étape par étape

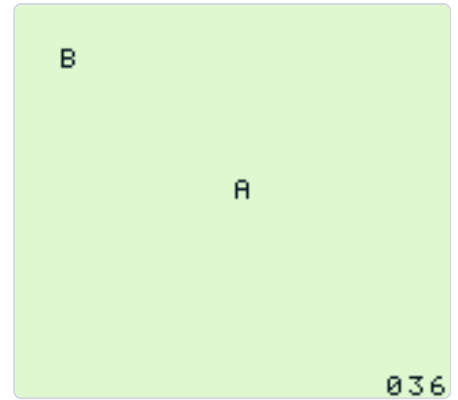
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



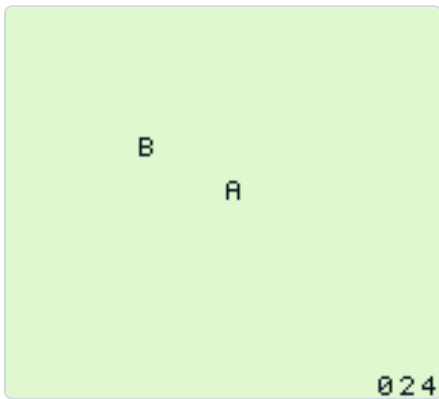
la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

Comme au 0.79, mais le B accélère toutes les 3 secondes ; en bas à droite, sa lenteur descend de 036 à 012.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ au départ, 36 images entre deux pas
- ✓ elle diminue avec le temps (36 → 24)
- ✓ elle s'arrête à 12, le plus rapide
- ✓ immobile, le A finit attrapé

Ensuite : **0.80. x et y rangés ensemble : struct** — niveau 0 sur 10. · gameboy3 — écrire une cartouche Game Boy en C++